

Flujo de trabajo del equipo (PR + Merge)

Objetivo: que todos trabajemos igual y el historial de main quede limpio.

Pasos básicos

1. Actualizar main antes de empezar

- `git checkout main`
- `git pull`

2. Crear una nueva rama de trabajo

¿Qué significa `git checkout -b feature/ABC-123-descripcion`?

Este comando se usa para **crear una nueva rama** y moverte a ella inmediatamente.

- `git checkout -b` → *crear rama y cambiarme a esa rama.*
- `feature/ABC-123-descripcion` → es el nombre de la nueva rama.

 **Importante:** este comando **no se usa con frecuencia**.

- Se utiliza **solo al inicio de una tarea nueva**, cuando necesitas crear una rama desde main.
- En el día a día, la mayoría de las veces trabajarás en ramas ya creadas (con `git checkout rama-existente`).

3. Hacer cambios, add y commit en tu rama.

- Editás **archivos** en tu rama.
- `git add.` → marcás cambios para commit.
- `git commit -m "mensaje"` → guardás los cambios en el historial de esa rama

4. Subir la rama al remoto

- `git push -u origin feature/ABC-123-descripcion`
- Cuando ya tenés commits en tu rama, tenés que mandarla al servidor (GitHub/GitLab).

Subir por primera vez la rama

`git push -u origin feature/ABC-123-descripcion`

- `origin` → es el remoto (GitHub/GitLab).
- `-u` → vincula tu rama local con la remota (solo la primera vez).

- feature/ABC-123-descripcion → es el nombre de tu rama.

Después, solo usarás:

git push

🔗 porque ya está vinculada.

5. **Abrir un Pull Request** hacia main.

Una vez subida la rama, vas a la plataforma (GitHub/GitLab/Bitbucket):

- Te va a sugerir crear un **PR**.
- Elegís como destino `main`.
- Escribís un título y descripción (qué hiciste, qué problema resuelve).
- Enviás el PR para que lo revisen.

6. **Revisión del PR:** al menos un compañero debe aprobar.

7. **Hacer merge** en la plataforma (GitHub/GitLab) usando:

- a. **Merge Commit** (si queremos ver todos los commits).
- b. **Squash & Merge** (si queremos un único commit).

8. **Borrar la rama** en remoto y en local.

Cuando ya hiciste el **Pull Request** y la rama fue **mergeada a main**, esa rama **ya cumplió su función**.

Si no la borrás:

Quedan un montón de ramas viejas en el repositorio remoto → difícil de encontrar las que sí están activas.

Se acumula “basura” y el repo se vuelve más difícil de mantener.

Por eso:

En remoto (GitHub/GitLab): la borramos para mantener el repo ordenado.

En local (tu computadora): la borramos para que no quede ocupando espacio.

📌 En **GitHub/GitLab**

→ clic en *Delete branch*.

📌 En local:

git branch -d feature/ABC-123-descripcion

◆ ¿Cómo se hace?

Después del merge borrar,

- en remoto desde GitHub (botón *Delete branch*).
- En tu máquina (local):

```
git branch -d feature/ABC-123-descripcion
```

-d = delete (borrar).

Si la rama aún no fue mergeada, Git no te dejará borrarla con -d (para evitar pérdida de trabajo).

👉 Regla de oro: **nunca pushear a main directo**. Todo va por PR.

📌 Comandos del día a día

Objetivo: tener a mano los comandos más usados para trabajar en el repositorio todos los días sin romper nada.

◆ 1. Actualizar tu rama antes de empezar a trabajar

```
git pull --rebase origin main
```

📖 Qué hace:

- Descarga los últimos cambios de main y los “reproduce” encima de tu trabajo.
- Evita commits de “merge” innecesarios y mantiene un historial limpio.

⚠️ Usar siempre este comando antes de empezar a trabajar cada día.

◆ 2. Cambiar de rama

```
git checkout nombre-de-rama
```

📖 Qué hace:

- Te mueve a otra rama en tu computadora.
- Si no existe, mostrará error.

🔑 **Tip:** Asegúrate de tener tu trabajo guardado antes de cambiar de rama.

Antes de hacer `git checkout otra-rama` tenés que:

1. Guardar tus archivos en el editor.
2. Confirmar con `git status` si hay cambios pendientes.
3. Si hay cambios, hacer `git add . && git commit -m "mensaje"`

◆ 3. Crear una nueva rama desde main

```
git checkout -b feature/ABC-123-descripcion
```

📖 Qué hace:

- Crea una rama nueva y te posiciona en ella.
- Usar solo cuando inicias una tarea nueva.

◆ 4. Ver en qué rama estás y el estado del repo

```
git status
```

📖 Qué hace:

- Muestra:
 - En qué rama estás trabajando.
 - Archivos modificados.
 - Archivos listos para commit.

◆ 5. Agregar cambios al área de preparación

```
git add archivo  
git add.
```

📖 Qué hace:

- Marca los archivos modificados para incluirlos en el próximo commit.
- `git add.` agrega todos los cambios en la carpeta actual.

◆ 6. Guardar cambios en tu rama (commit)

```
git commit -m "Mensaje claro sobre lo que hiciste"
```

📖 Qué hace:

- Guarda los cambios en el historial local de tu rama.
 - 🔑 **Tip:** Usa mensajes claros y concisos.

◆ 7. Subir tus cambios al remoto

```
git push
```

📖 Qué hace:

- Envía los cambios de tu rama al servidor.
- Si es la primera vez que subís una rama, usá:

```
git push -u origin feature/ABC-123-descripcion
```

◆ 8. Ver historial de commits

```
git log --oneline --graph --decorate
```

📖 Qué hace:

- Muestra los commits de forma resumida y en árbol.
- Ideal para entender el historial.

◆ 9. Ver ramas disponibles

```
git branch  
git branch -r  
git branch -a
```

📖 Qué hace:

- `git branch`: ramas locales.
- `git branch -r`: ramas remotas.
- `git branch -a`: todas las ramas (locales y remotas).

◆ 10. Eliminar ramas

```
git branch -d nombre-rama  
git push origin --delete nombre-rama
```

📖 Qué hace:

- `-d`: borra la rama localmente (si ya está mergeada).
- `--delete`: borra la rama en el remoto.

◆ 11. Ver qué archivos cambiaste

```
git diff
```

📖 Qué hace:

- Muestra los cambios hechos en archivos que todavía no agregaste con `git add`.

◆ 12. Revertir un archivo al último commit

```
git checkout -- nombre-archivo
```

Qué hace:

- Revierte el archivo al estado del último commit.
 **Atención:** Se pierden cambios no guardados.

Cómo resolver conflictos de merge

 **Objetivo:** Poder combinar ramas sin romper nada, manteniendo el historial limpio.

1. Cuando pasa esto

Un **conflicto de merge** aparece cuando Git no puede decidir automáticamente qué versión de un archivo mantener porque **dos personas cambiaron la misma parte del código** en ramas diferentes.

Vas a ver un mensaje como:

CONFLICT (content): Merge conflict in archivo.js

Automatic merge failed; fix conflicts and then commit the result.

2. Detectar los archivos en conflicto

git status

Qué hace:

- Muestra qué archivos tienen conflictos (both modified).

3. Abrir los archivos y buscar las marcas de conflicto

En el archivo vas a encontrar algo así:

```
<<<<<<< HEAD
```

```
console.log("Versión en tu rama");
```

```
=====
```

```
console.log("Versión en main");
```

```
>>>>>>> main
```

- Todo lo que está entre <<<<<<< HEAD y ===== es **tu versión**.
- Todo lo que está entre ===== y >>>>>>> main es **la versión de la otra rama**.

◆ 4. Resolver el conflicto manualmente

Cuando ves esto en tu archivo:

```
<<<<<<< HEAD
console.log("Versión en tu rama");
=====
console.log("Versión en main");
>>>>>>> main
```

Git te está mostrando **dos versiones**:

- **HEAD**: la rama donde estás parado (tu trabajo actual).
- **main** (o el nombre que aparezca): la rama con la que intentás hacer merge/rebase.

Opción 1: Elegir tu versión

Si querés conservar solo tu código:

```
console.log("Versión en tu rama");
```

Opción 2: Elegir la versión de la otra rama

Si querés conservar solo el código de la otra rama:

```
console.log("Versión en main");
```

Opción 3: Combinar ambas versiones

Si ambas partes tienen cambios útiles:

```
console.log("Versión en tu rama");
```

```
console.log("Versión en main");
```

◆ Limpieza obligatoria

- **Eliminá todas las marcas** (<<<<<<<, =====, >>>>>>>).
- Guardá el archivo.

💡 **Tip para hacerlo más fácil:**

Si usás **VS Code**, Git te resalta los conflictos y te da botones como:

- *Accept Current Change* (aceptar tu versión)
- *Accept Incoming Change* (aceptar la otra versión)
- *Accept Both Changes* (combinar)

Así evitás tocar las marcas manualmente.

◆ 5. Marcar el conflicto como resuelto

git add archivo.js

Qué hace:

- Indica a Git que el archivo ya fue corregido.

◆ 6. Continuar el merge o rebase

Si estabas haciendo un **merge**:

git commit

Si estabas haciendo un **rebase**:

git rebase --continue

◆ 7. Verificar que todo esté bien

git status

- Si no hay más conflictos, tu merge/rebase estará completo.

◆ 8. Subir los cambios

git push

Qué hace:

- Actualiza la rama remota con los cambios ya resueltos.

Comandos avanzados: solución de problemas

 **Objetivo:** Tener a mano comandos útiles cuando las cosas no salen como esperabas.

◆ 1. Guardar cambios sin hacer commit (Stash)

git stash

git stash list

git stash pop

Qué hace:

- git stash: guarda cambios sin hacer commit, ideal si necesitas cambiar de rama urgente.

- `git stash list`: muestra los cambios guardados.
- `git stash pop`: aplica el último stash y lo elimina de la lista.

💡 Ejemplo:

1. Estás en mitad de algo, pero debes cambiar de rama:
 - `git stash`
 - `git checkout otra-rama`
2. Volvés a tu rama y recuperás tus cambios:
 - `git checkout mi-rama`
 - `git stash pop`

◆ 2. Deshacer commits o cambios (Reset)

- `git reset --soft HEAD~1`
- `git reset --hard HEAD~1`

📖 Qué hace:

- `--soft`: mueve el puntero del commit atrás, **pero conserva cambios en el área de stage**.
- `--hard`: vuelve el repo al commit anterior **eliminando cambios** (⚠️ cuidado, se pierden).

💡 Cuándo usarlo:

- Para deshacer commits recientes que no deberían existir.

◆ 3. Recuperar commits borrados (Reflog)

`git reflog`

📖 Qué hace:

- Muestra el historial de todos los movimientos (incluye commits eliminados).
- Permite volver atrás incluso después de un `reset --hard`.

💡 Ejemplo:

`git checkout HEAD@{2}`

Vuelve a tu repo al estado de hace 2 pasos.

◆ 4. Aplicar un commit específico a otra rama (Cherry-pick)

`git cherry-pick <hash-del-commit>`

📖 Qué hace:

- Copia un commit específico de una rama y lo aplica a otra.
 - Útil para traer un fix puntual sin mergear todo.
-

◆ 5. Eliminar cambios no guardados

git checkout -- archivo

📖 Qué hace:

- Revierte un archivo a su último commit.
⚠️ **Cuidado:** Los cambios no guardados se pierden.

📖 Definiciones a Tener en Cuenta (Git y Flujo de Trabajo en Equipo)

◆ Repositorio (repo)

Es la “carpeta de proyecto” que Git controla. Puede estar en tu compu (local) o en un servidor como GitHub (remoto).

◆ Rama (branch)

Una “línea de trabajo paralela” dentro del repo.

Ejemplo: main (rama principal), feature/ABC-123 (rama de una tarea).

◆ main

La rama principal. Representa la versión estable del proyecto.

👉 Regla de oro: nunca pushear directo a main.

◆ checkout

Comando para moverte de una rama a otra.

Ejemplo: git checkout nombre-rama.

◆ commit

Un “punto de guardado” en el historial de tu rama. Incluye los cambios y un mensaje que explica qué hiciste.

Ejemplo: git commit -m "arreglo bug en login".

◆ add

Prepara los archivos que querés incluir en el próximo commit.

Ejemplo: git add archivo.js o git add ..

◆ push

Envía tus commits locales al repositorio remoto (GitHub/GitLab).

Ejemplo: git push.

◆ pull

Descarga cambios del remoto hacia tu repo local.

Ejemplo: git pull.

◆ **pull --rebase**

Descarga cambios de main y “reproduce” tus commits encima.

👉 Mantiene un historial limpio sin merges innecesarios.

◆ **merge**

Combina los cambios de una rama en otra.

Ejemplo: cuando un PR se aprueba y se integra a main.

◆ **PR (Pull Request)**

Solicitud para que los cambios de tu rama se revisen y se integren a main.

👉 Es el corazón del trabajo en equipo con Git.

◆ **origin**

Nombre por defecto del repositorio remoto (normalmente en GitHub/GitLab).

◆ **stash**

Caja temporal donde podés guardar cambios sin hacer commit.

👉 Sirve si tenés que cambiar de rama sin perder lo que estabas haciendo.

Ejemplo: git stash, git stash pop.

◆ **reflog**

Registro oculto de todos los movimientos en tu repo.

👉 Último recurso para recuperar cosas borradas.

◆ **hash del commit**

Identificador único (como un “DNI”) de cada commit.

Ejemplo: a1b2c3d4.